Experiment 4

Student Name: Muskan

UID:23BAI70172

Branch: BE-AIT-CSE

Section/Group:23AIT_KRG 1A

Semester:6th

Date of Performance: 4,feb 2026

Subject Name: Full STack - II

Subject Code:23CSH-382

1. **Title:**

   Redux Toolkit & Asynchronous State Management in EcoTrack

2. **Aim:**

   To optimize the performance of the EcoTrack React application using memoization techniques and code splitting, and to enhance the user interface using enterprise-grade Material UI components.

3. **Objective:**

After completing this experiment and its follow-up tasks, the student will be able to:

1. Understand the causes of unnecessary re-renders in React applications
2. Optimize React components using React.memo to prevent avoidable re-renders
3. Apply useMemo to efficiently compute derived data and avoid redundant calculations
4. Use useCallback to memoize event handler functions and improve component performance
5. Implement lazy loading of components and routes using React.lazy and Suspense
6. Reduce initial bundle size and improve application load performance through code splitting
7. Enhance the visual appearance and usability of the EcoTrack application using Material UI components
8. Design a clean, consistent, and responsive user interface using Material UI layouts and typography

code :

logslice.jsx

periment2 > navigator > src > stores > JS logSlice.js > logSlice > initialState

```javascript
import { createSlice, createAsyncThunk } from "@reduxjs/toolkit";

export const fetchLogs = createAsyncThunk(
    "logs/fetchLogs",
    async() => {
        await new Promise((resolve) => setTimeout(resolve, 1000));

        return  [
            { id: 1, activity: "Car Travel", carbon: 4 },
            { id: 2, activity: "Electricity Usage", carbon: 6 },
            { id: 3, activity: "Cycling", carbon: 0 },
        ]
    }
)

const logSlice = createSlice({
    name : "logs",
    initialState : {
        data: [],
        status: "idle",
        error: null,
    },
    reducers: {},
    extraReducers: (builder) => {
        builder.addCase(fetchLogs.pending, (state, action) => {
            state.status = "pending";
        })
        .addCase(fetchLogs.fulfilled, (state, action) => {
            state.status = "success";
            state.data = action.payload;
        })
        .addCase(fetchLogs.rejected, (state, action) => {
            state.status = "failed";
            state.error = action.error.message;
        })
    }
}
)

export default logSlice.reducer;
```

logs.jsx

erriment2 > navigator > src > pages > Logs.jsx > Logs

```jsx
import { useEffect,useMemo,useCallback } from "react";
import { useDispatch, useSelector } from "react-redux";
import { fetchLogs } from "../stores/logSlice";
//import { calculateTotalCarbon } from '../data/Calculation.js';

function Logs() {
  const dispatch = useDispatch();

  // 1. Redux store se data, status aur error nikalna
  const { data, status, error } = useSelector((state) => state.logs);

  // 2. useEffect use karke data fetch karna
  useEffect(() => {
    if (status === "idle") {
      dispatch(fetchLogs());
    }
  }, [status, dispatch]);

  const totalCarbon = useMemo(()=>{
    return data.reduce((acc,log)=> acc + log.carbon, 0);
  },[data]);

  // 3. Loading aur Error handling
  if (status === "pending") {
    return <p style={{ textAlign: 'center', marginTop: '50px', fontSize: '40px', color: 'white' }}>Loading Logs ......</p>;
  }

  if (status === "failed") {
    return <p style={{ color: 'red', textAlign: 'center' }}>Error: {error}</p>;
  }


  //const totalCarbon = data ? calculateTotalCarbon(data) : 0;

  return (
    <div style={{ color: 'black', background: 'white', height: '100%',width:'700px', border: '5px solid black', boxShadow: '0 8px 18px rgba(251, 246, 246, 0.2)', marginL
      <h1>Total Carbon Footprint: {totalCarbon}</h1>

      <h2>Activity Logs</h2>
      <table style={{ flex: 1, border: "2px solid black", margin: "auto", textAlign: "center", width: "100%" }}>
        <thead>
          <tr>
            <th style={{ border: "2px solid black" }}>Activity</th>
            <th style={{ border: "2px solid black" }}>Carbon Footprints</th>
          </tr>
        </thead>
        <tbody>
          {/* Ab 'logs' ki jagah Redux ka 'data' map hoga */}
          {data.map((log) => (
            <tr key={log.id} style={{ color: 'white', backgroundColor: 'blue' }}>
              <td>{log.activity}</td>
              <td>{log.carbon}kg</td>
            </tr>
          ))}
        </tbody>
      </table>

      {/* Filtered Logs: Less than or equal to 4kg */}
      <h2 style={{ marginTop: "100px" }}>Filtered Logs less than equal to 4kg</h2>
      <table style={{ flex: 1, border: "2px solid black", margin: "auto", width: "100%" }}>
        <thead>
          <tr>
            <th style={{ border: "2px solid black", textAlign: "center" }}>Activity</th>
            <th style={{ border: "2px solid black", textAlign: "center" }}>Carbon Footprints</th>
          </tr>
```

Restart V

```
      <thead>
        <tr>
          <th style={{ border: "2px solid black", textAlign: "center" }}>Activity</th>
          <th style={{ border: "2px solid black", textAlign: "center" }}>Carbon Footprints</th>
        </tr>
      </thead>
      <tbody>
        {data.filter((log) => log.carbon <= 4).map((log) => (
          <tr key={log.id} style={{ color: 'white', backgroundColor: 'green' }}>
            <td>{log.activity}</td>
            <td>{log.carbon}kg</td>
          </tr>
        ))}
      </tbody>
    </table>

    {/* Filtered Logs: Greater than 4kg */}
    <h2 style={{ marginTop: "100px" }}>Filtered Logs greater than 4kg</h2>
    <table style={{ flex: 1, border: "2px solid black", margin: "auto", width: "100%" }}>
      <thead>
        <tr>
          <th style={{ border: "2px solid black", textAlign: "center" }}>Activity</th>
          <th style={{ border: "2px solid black", textAlign: "center" }}>Carbon Footprints</th>
        </tr>
      </thead>
      <tbody>
        {data.filter((log) => log.carbon > 4).map((log) => (
          <tr key={log.id} style={{ color: 'white', backgroundColor: 'red' }}>
            <td>{log.activity}</td>
            <td>{log.carbon}kg</td>
          </tr>
        ))}
      </tbody>
    </table>
  </div>
);
}

export default Logs;
```

Store.jsx

Experiment2 > navigator > src > stores > JS store.js > default

```js
import {configureStore} from "@reduxjs/toolkit";
import logsReducer from "./logSlice";

const store = configureStore({
    reducer: {
        logs : logsReducer,
    },
});

export default store;
```

Auth Context .jsx

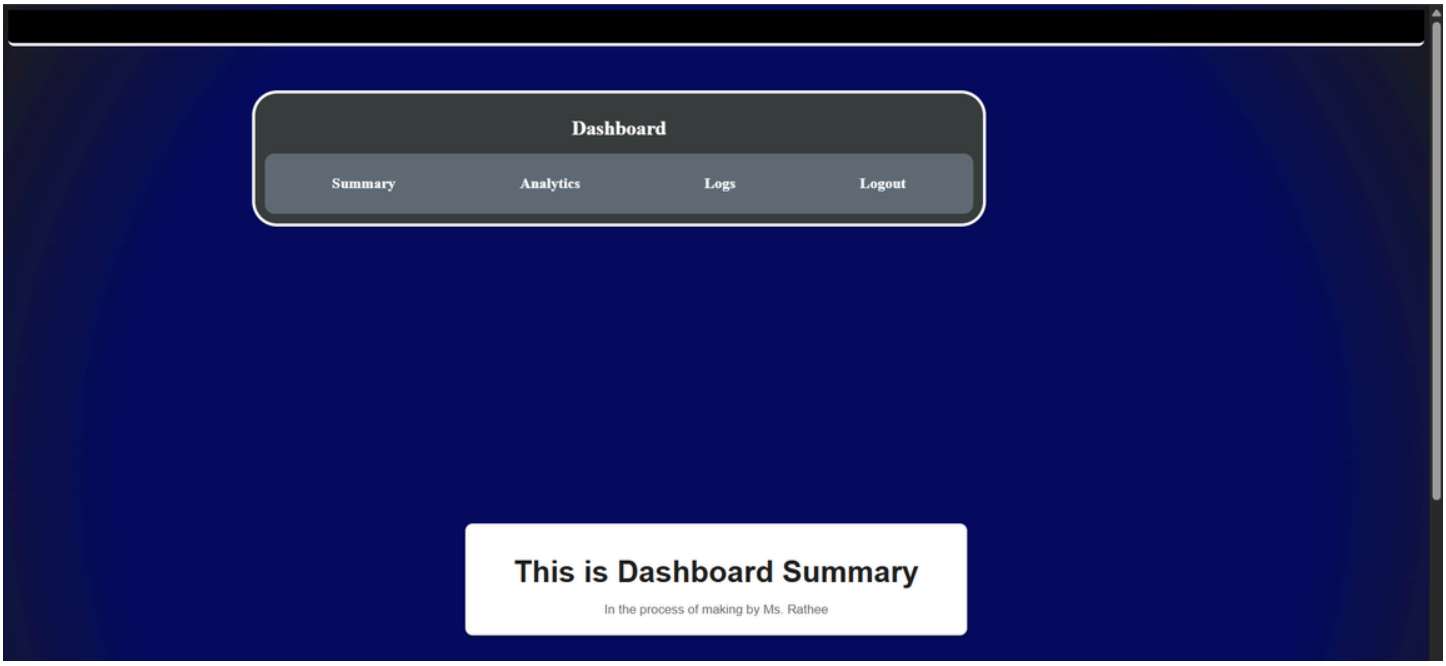
Experiment2 > navigator > src > context > AuthContext.jsx > ...

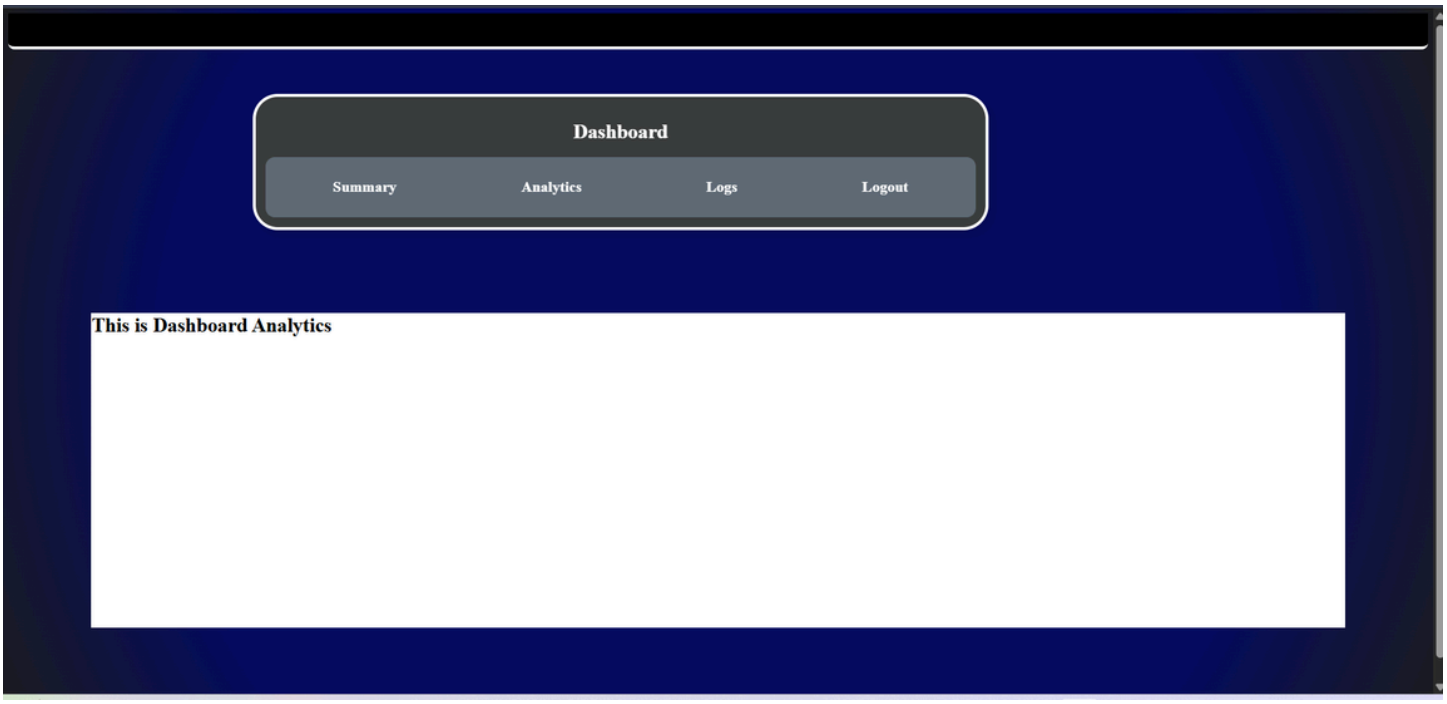
```jsx
import { createContext, useContext, useState} from "react";
 //Empty context global state variaable initially empty
const AuthContext = createContext(null);

//passing it to the children components
//what awe are passing isAuthenticated and setIsauthenticated
export const AuthProvider = ({children}) => {
    const [isAuthenticated,setIsAuthenticated] = useState(false);
//returning the provider
    return(
        <AuthContext.Provider value={
            {isAuthenticated,setIsAuthenticated}
        }>
            {children}
            </AuthContext.Provider>
    )
}

export const useAuth = () => useContext(AuthContext);
```

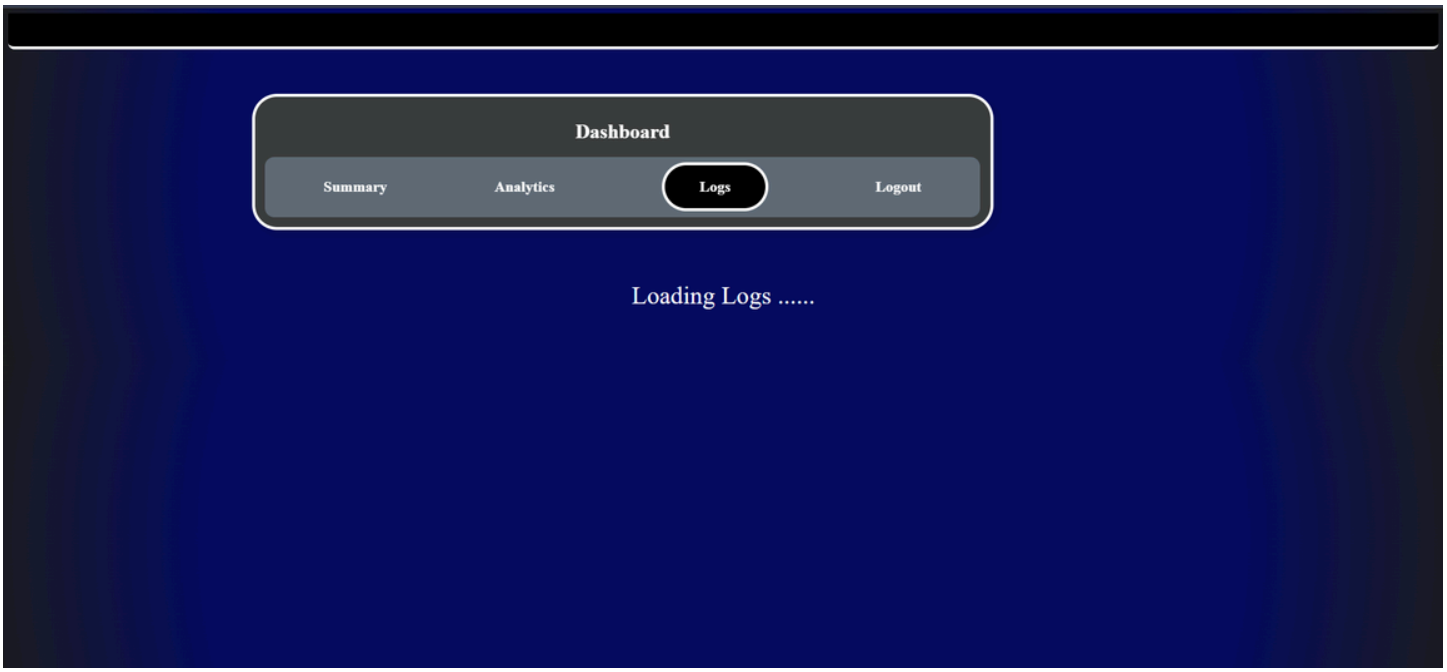
DashboardLayout.jsx

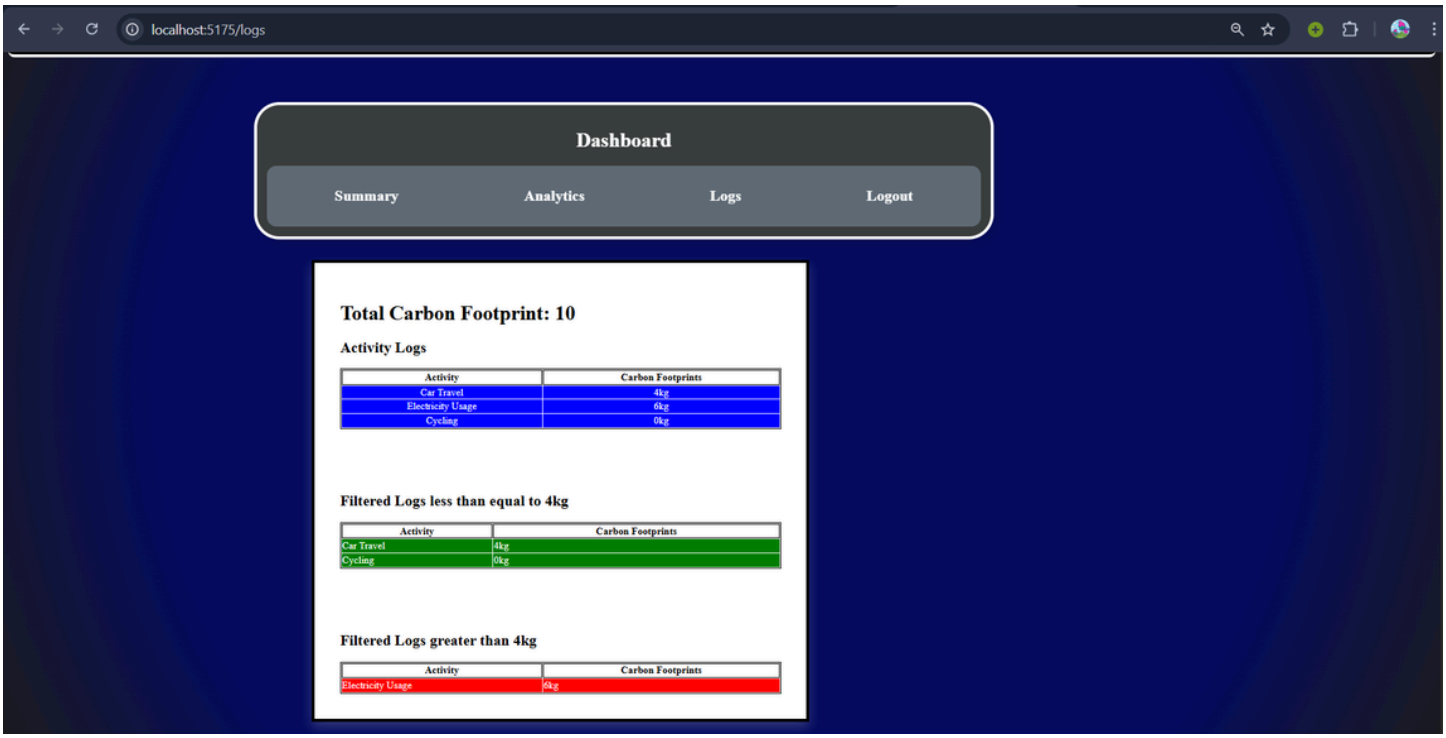
```jsx
import {Link,Outlet} from 'react-router-dom';
import './DashBoardLayout.css';

const DashboardLayout = () => {
    return(
        <div className = "dashboard-container">
            <div className="dashboard-header">
            <h1>Dashboard</h1>

            <nav className="dashboard-nav">
                    <Link to="summary" className="nav-link"><h2>Summary</h2></Link>
                     <Link to='analytics' className="nav-link"><h2>Analytics</h2></Link>
                     <Link to='logs' className='nav-link'><h2>Logs</h2></Link>
                     <Link to='logout' className='nav-link'><h2>Logout</h2></Link>

            </nav>

            </div>


        <div className="dashboard-content">
            <Outlet />
        </div>

      </div>
    )
}
export default memo(DashboardLayout);
```

Output:




Dashboard
Summary
Analytics
Logs
Logout
This is Dashboard Analytics


Dashboard
Summary
Analytics
Logs
Logout
Loading Logs .....



.Learning Outcome

- We learnt about React Apps and how to create them.
- We learnt about redux and it's components.